ENGINEERING LEDGER · SESSION RECORD

RetailApp — Cycle 2

Multi-tenant company & subscription platform, FIFO cost-layer inventory, billing, and transaction reversal — built on top of the Cycle 1 single-tenant ledger app.

Go 1.25 · chi · pgx/sqlc · Postgres 16
React 19 · Vite · Tailwind v4
Written 2026-09-07

What RetailApp is

RetailApp is a wholesale/distributor ledger: a company buys stock from factories, sells it to shops, and tracks who owes what. Cycle 1 was a single tenant — one login set, one set of products, one shared database. Cycle 2 turns it into a small multi-tenant SaaS: a platform operator (**SUPER_ADMIN**) provisions **companies**, each company runs its own isolated catalog, purchases, sales and staff, and each company's access is gated by a **subscription** (trial or paid) that the platform operator grants and extends.

BACKEND

Go

chi router, pgx/v5, sqlc-generated queries

FRONTEND

React

TypeScript, Vite, Tailwind v4

STORAGE

Postgres

one DB, company_id on every row

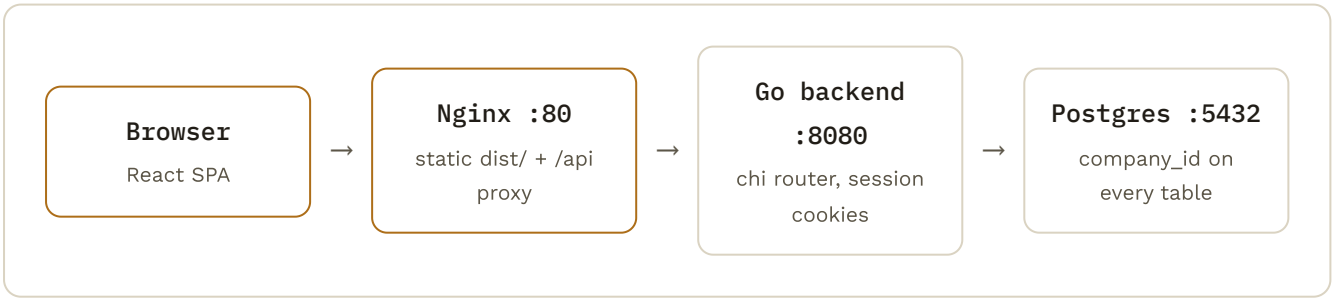
THIS CYCLE

10 phases

foundation → product → billing → UI

Architecture

One process, one database. Nginx is the only thing the internet touches in production; the Go binary and Postgres both listen on localhost only.



Every company-scoped query takes `company_id` from the authenticated session — never from the request body or URL — so changing an id in a request can never reach another company's row. Verified directly: a product created under Company A returns 404 when Company B asks for it by id.

Authorization hierarchy

```
SUPER_ADMIN # company_id = NULL, platform-level
└─ COMPANY (e.g. "ABC Traders")
   └─ COMPANY_ADMIN # full access, always passes permission checks
      └─ STAFF
         └─ purchase_access # gates POST /api/purchases
         └─ sales_access # gates POST /api/sales
         └─ sales_below_cost_approve # gates selling under cost
```

Folded into one `users` table (`company_id` nullable, `user_type`, three boolean access flags) rather than a separate membership table — there's no requirement for one login to belong to more than one company, so a join table would have been indirection with no payoff.

Phase-by-phase summary

Sequenced so the multi-tenant foundation lands first — every later phase's queries and handlers depend on `company_id` scoping and the permission model existing already.

`phase 0`	Companies, subscriptions, the SUPER_ADMIN/COMPANY_ADMIN/STAFF hierarchy, and `company_id` retrofitted onto every existing table and query.
`phase 1`	Product units (fixed list), category/subcategory, per-company-unique SKU, and FIFO cost layers so the same product can carry multiple buying prices over time.
`phase 2`	Primary/secondary contact numbers on factories and shops.
`phase 3`	Below-cost sale warning, computed from the real FIFO cost (not a static field), gated behind a `sales_below_cost_approve` permission.
`phase 4`	Sequential bill numbers, server-rendered PDF bills (`go-pdf/fpdf`), print/download/share from the browser.
`phase 5`	CANCEL-based reversal for sales and purchases — exact cost-layer restoration, stock reversal, mandatory reason, full audit trail.

phase 6 Responsive layout — collapsible drawer nav, horizontally-scrolling tables, touch-sized controls.

phase 7 Super Admin console — create a company (with its first Company Admin), grant a trial, activate or extend a subscription.

phase 8 Company Admin console — create staff, toggle purchase/sales/below-cost permissions, disable a login.

phase 9 Go tests for the highest-risk logic: multi-company isolation, FIFO costing math, cancellation precision, subscription extension.

Data model

New or materially changed tables. Every business table below carries `company_id` .

TABLE	PURPOSE	NOTABLE COLUMNS
companies	One row per tenant	`company_code` **unique**, `status`, `joining_date`
subscriptions	A company's access-period history	`subscription_type`, `start/expiry_date`, `status`, `granted_by`
users	Every login, all three roles	`company_id` **nullable**, `user_type`, `purchase/sales/below-cost flags`
categories / subcategories	Product grouping, per company	`subcategory.category_id` → `categories`
units	Global fixed unit list	`PIECE`, `BOX`, `PACK`, `KG`, `GRAM`, `LITRE`, `ML`, `METER`, `SET`, `DOZEN`
products	Catalog	`sku` **unique per company**, `current_selling_price`, `current_stock`
product_cost_layers	One row per purchase — the FIFO ledger	`buying_price`, `quantity_received`, `quantity_remaining`, `status`

sale_item_cost_consumptions	Exactly which layer(s) a sale drew from	`sale_item_id, product_cost_layer_id, quantity, unit_cost`
purchases / sales	Bills	`bill_number` **`unique per company`**`, status, cancelled_reason/by/at`
bill_sequences	Per-company counter feeding bill_number	`(company_id, bill_type) → next_number`
audit_log	Who cancelled what, and why	`action, entity_type/id, reason, inventory_impact jsonb`

Design decisions worth remembering

DECISION	WHY
SKU unique *per company*, not globally	Two tenants can both use "NIKE-001" — enforced as a composite DB constraint.
Factories/shops stay separate tables	Less invasive than a generic `party` table; nothing needs to treat suppliers and customers polymorphically.
Bill numbers from a real counter row	`bill_sequences` incremented inside the bill's own transaction — dense, per-company, portable.
Expired company: reads allowed, writes blocked	An expired customer can still see their data; new business activity is what's gated (402 Payment Required).
Purchase cancel blocked if partially sold	If any of that purchase's cost-layer quantity was already sold, cancellation is refused (409) rather than going negative.
Cancellation restores the *exact* layer, never re-derives FIFO	A later purchase can add a new layer before a sale gets cancelled — re-running FIFO at cancel-time could touch the wrong layer.

API — Auth

METHOD	PATH	AUTH	NOTES

POST	/api/auth/login	public	{username, password} → sets session cookie, returns user incl. user_type/company_id/permission flags
POST	/api/auth/logout	public	Clears the session cookie
GET	/api/auth/me	public	401 if not logged in, else current user

API — Product catalog

METHOD	PATH	AUTH	NOTES
GET	/api/units	any user	Fixed reference list (10 seeded units)
GET	/api/categories	any user	Company's categories
POST	/api/categories/	subscription	{name}
GET	/api/categories/{id}/subcategories	any user	
POST	/api/categories/{id}/subcategories	subscription	{name}
GET	/api/products/	subscription	Company's catalog
POST	/api/products/	subscription	{name, sku, unit, categor subcategory_id?, selling_price}
GET	/api/products/{id}	subscription	
PUT	/api/products/{id}	subscription	Editing selling_price neve rewrites past sale_items
DELETE	/api/products/{id}	subscription	
GET	/api/products/{id}/cost	any user	Oldest active cost-layer price, for the frontend's inline below-cost hint

API — Parties

METHOD	PATH	AUTH	NOTES

GET	/api/factories/	subscription	
POST	/api/factories/	subscription	{name, contact_person, primary_phone, secondary_phone?, address}
GET	/api/factories/{id}	subscription	
PUT	/api/factories/{id}	subscription	
DELETE	/api/factories/{id}	subscription	
GET	/api/shops/…	subscription	Same shape as factories
POST / PUT / DELETE	/api/shops/…	subscription	primary_phone required, secondary_phone optional & must differ

API — Purchases & sales

METHOD	PATH	AUTH	NOTES
GET	/api/purchases/	subscription	
POST	/api/purchases/	purchase_access	{factory_id, invoice_no, amount_paid, items[], new_selling_price?}. Each item creates a new cost layer.
GET	/api/purchases/{id}/items	subscription	
POST	/api/purchases/bills/{id}/cancel	purchase_access	{reason} required. 409 if ar of its stock was already so
GET	/api/sales/	subscription	
POST	/api/sales/	sales_access	{shop_id, payment_type, amount_paid, items[]}. 403 any line is below FIFO cost and the actor lacks below-cost approval.
GET	/api/sales/{id}/items	subscription	
POST	/api/sales/bills/{id}/cancel	sales_access	{reason} required. Restores

			the exact cost-layer quantities the sale consumed.

API — Bills

METHOD	PATH	AUTH	NOTES
GET	/api/sales/bills/{id}	subscription	JSON bill (company, counterparty, line items, totals) for the on-screen view
GET	/api/sales/bills/{id}/pdf	subscription	Same data, rendered as a PDF
GET	/api/purchases/bills/{id}	subscription	
GET	/api/purchases/bills/{id}/pdf	subscription	

API — Payments & reports

METHOD	PATH	AUTH	NOTES
GET / POST	/api/payments/	subscription	{party_type: shop|factory, party_id, amount, payment_mode, notes}
GET	/api/shops/{id}/balance	subscription	Opening balance + sales − paid − payments
GET	/api/factories/{id}/balance	subscription	Purchases − paid − payments
GET	/api/reports/dashboard	subscription	Today's sales/purchases, profit, low stock, dues, payables
GET	/api/reports/shop-dues	subscription	
GET	/api/reports/factory-payables	subscription	
GET	/api/reports/low-stock	subscription	

API — Company & staff

METHOD	PATH	AUTH	NOTES
GET	/api/company/subscription-status	any user	{status, expiry_date, days_remaining, warning_level}
GET	/api/company/users	company admin	List staff
POST	/api/company/users	company admin	{name, username, password, purchase_access, sales_access, sales_below_cost_approve}
PUT	/api/company/users/{id}/permissions	company admin	{purchaseAccess, salesAccess, salesBelowCostApprove}
PUT	/api/company/users/{id}/disable	company admin	Disabled users can't log in

API — Super admin

METHOD	PATH	AUTH	NOTES
POST	/api/super-admin/companies	super admin	{company_name, company_code, admin admin_username, admin_password} — c the company + its firs Company Admin toget
GET	/api/super-admin/companies	super admin	All companies + latest subscription status
GET	/api/super-admin/companies/{id}	super admin	
POST	/api/super-admin/companies/{id}/trial	super admin	One month from toda
POST	/api/super-admin/companies/{id}/subscription	super admin	{subscription_type, an months}